

Assignment 3

Question 1: You are building a landing page for a small bakery in your city. The page looks fine on your laptop but when you open it on your phone, the text is too small and the images go outside the screen. Which CSS technique will you use to fix this and how will you start?

Answer:

The technique we use here is 'Responsive Web Design'.

1. Include this meta tag in <head>: <meta name="viewport" content="width=device-width, initial-scale=1.0">
2. On the tag provide CSS:

```
img {  
    max-width: 100%;  
    height: auto;  
}
```

Question 2: You added a navbar at the top of your website. The client wants the navbar to always stay visible at the top, even when the user scrolls down. Which CSS property will you use and what value will you set?

Answer:

CSS property to be used here is 'position'. We can do this with either 'fixed' or 'sticky' value.

1.

```
position: fixed;  
top: 0;  
width: 100%;  
z-index: 1000;
```
2.

```
position: sticky;  
top: 0;  
width: 100%;  
z-index: 1000;
```

Just remember to provide top margin to body when using method 1 to push the contents out from below the nav bar.

Question 3: You wrote CSS for a button — first you added a class .btn with blue background, then added an id #submitBtn with red background to the same button. The button is showing red instead of blue. Why is this happening?

Answer:

This is happening due to CSS Specificity rules. In CSS, selectors have different weights. An ID selector (`#submitBtn`) has a higher specificity weight than a class selector (`.btn`). Because the ID selector is more specific, the browser applies its style (red background) over the class style (blue background), regardless of the order they were written in the stylesheet.

Question 4: You are building a product card for an online store. You want three cards to sit side by side on desktop, but stack one below the other on mobile. Which layout system will you use and how will you make it responsive?

Answer:

We can achieve this with either CSS Flexbox or CSS Grid.

Using Flexbox: Set the container to `display: flex;` and `flex-wrap: wrap;`. Give the cards a base width of 100% for mobile, and use a media query for desktops to set their width to roughly 33.33% (e.g., `flex: 0 0 30%;` with spacing).

Using Grid: You can achieve this cleanly without media queries using `grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));`. This will naturally stack the cards on mobile when the screen drops below 300px per card, and align them side-by-side on desktop.

Question 5: You have a section with text and you want to add some space inside the section between the text and the border, and also some space outside the section to separate it from the next section. Which two CSS properties will you use and how will they differ?

Answer:

The properties to be used are: padding and margin.

Their difference:

- padding: Adds space inside the section, between the content (text) and its border. It affects the inner background area of the element.
- margin: Adds space outside the section, pushing away neighboring elements. It creates a clearing outside the element's border.

Question 6: Your senior gives you a design where a 'SALE' label has to sit on the top-right corner of every product image. The label should overlap the image. Which positioning approach will you use?

Answer:

This can be done by using absolute positioning inside relative container:

1. Set the container holding the product image and the label to `position: relative;`

2. Set the 'SALE' label itself to position: absolute;
3. Use coordinates top: 0; and right: 0; to pin the label perfectly to the top-right corner, letting it overlap the image.

Question 7: What is the difference between display: none and visibility: hidden? Give one situation where you would use each.

Answer:

1. The Difference:
 - a. display: none removes the element entirely from the page layout. It takes up zero space (as if it doesn't exist).
 - b. visibility: hidden hides the element, but the element still takes up its original space in the layout (leaving an empty gap).
2. When to use each:
 - a. Use display: none when building a mobile hamburger menu that should be completely hidden and not take up space until the menu button is clicked.
 - b. Use visibility: hidden when you want an error message below a form input to disappear, but you don't want the form layout to shift or jump up when it vanishes.

Question 8: What is the difference between Flexbox and CSS Grid? When should you use each?

Answer:

Difference between the two:

- Flexbox is a one-dimensional layout system (deals with either rows *or* columns at a time). It aligns items dynamically based on content size.
- CSS Grid is a two-dimensional layout system (deals with *both* rows and columns simultaneously). It is based on a strict predefined grid template.

When to use each:

- Use Flexbox for smaller component layouts or linear structures, like a navigation bar, a row of buttons, or aligning an icon next to text.
- Use Grid for complex, full-page layouts, magazine-style designs, or card dashboards where elements must perfectly align horizontally and vertically.

Question 9: What are the three ways to add CSS to an HTML page? Which one is preferred for real projects and why?

Answer:

The Three Ways:

1. **Inline CSS:** Written directly inside the HTML element using the style attribute.
2. **Internal/Embedded CSS:** Written inside a <style> tag within the HTML document's <head>.
3. **External CSS:** Written in a separate .css file and linked to the HTML document via a <link> tag.

Among the above three, we prefer External CSS because it completely separates structure (HTML) from presentation (CSS), keeping code organised. It allows you to style multiple web pages with a single file, reducing redundancy, improving maintainability, and allowing browsers to cache the CSS file for faster page load speeds.

Question 10: What is a media query in CSS? Write a small example that changes the background color when the screen is smaller than 600px.

Answer:

A media query is a CSS technique that applies specific blocks of CSS rules only if certain device conditions are met—most commonly a device's viewport width. It is the foundation of responsive web design.

The following code shows how to do it (needs to be in a .css file or inside <style> tag in <head> of a .html file):

```
/* Default desktop background color */
```

```
body {  
    background-color: lightblue;  
}
```

```
/* Changes background to lightgreen when screen is smaller than 600px */
```

```
@media (max-width: 599px) {  
    body {  
        background-color: lightgreen;  
    }  
}
```

